KUBERNETES CKA PREP › MY LEARNING NOTES › SESSION 05

Namespace Quota & LimitRange

Topic: ResourceQuota + LimitRange | CKA Weekend Course · Video 9, 16th Sep | 2026-09-11

Namespace = Segmentation of the Cluster

A Namespace lets you divide a Kubernetes cluster for different applications or environments (e.g. `dev`, `test`, `prod`) so you can run and differentiate services per team/app/environment inside one physical cluster.

Giving a Namespace a ResourceQuota

BASH

```
kubectl create quota dev-quota --hard=cpu=2,memory=3G,pods=5,services=3 -n dev
# resourcequota/dev-quota created
```

This caps the **total** the `dev` namespace can consume: 2 CPU cores, 3G memory, max 5 pods, max 3 services.

BASH

```
kubectl describe ns dev
# Resource Quotas
#   Name:       dev-quota
#   Resource    Used   Hard
#   cpu         0      2
#   memory      0      3G
#   pods        1      5
#   services    0      3
```

> ⚠️ **The moment you set a quota, requests/limits become mandatory**
>
> Once a ResourceQuota exists on a namespace, **every pod created in it must explicitly define `resources.requests` and `resources.limits`** — Kubernetes can't count usage against a quota if it doesn't know how much a pod is asking for. A pod YAML with no `resources:` block gets rejected:

```bash
kubectl create -f mypod.yaml
# Error from server (Forbidden): error when creating "mypod.yaml": pods "webapp" is
forbidden:
# failed quota: dev-quota: must specify cpu for: nginx; memory for: nginx
```

> After adding a `resources` block with requests/limits, the pod creates successfully and the namespace's quota `Used` column updates.

Quota Enforcement on a Deployment's ReplicaSet

Deploying a 5-replica Deployment inside a namespace whose quota caps `pods=5` works exactly up to the cap — the 6th (or any pod beyond the cap) fails to create, and the failure surfaces as `ReplicaFailure` on the Deployment with repeated `FailedCreate` events on the ReplicaSet:

```bash
Type      Reason          Age   From                   Message
Warning   FailedCreate    26s   replicaset-controller   Error creating: pods "nginx-
deployment-...-zp56v" is
forbidden: exceeded quota: dev-quota, requested: pods=1, used: pods=5, limited: pods=5
```

Read that event message literally — it's ReplicaSet trying (and failing) to create the pod, not some vague scheduling error, and the exact quota values are printed right there.

The Gap ResourceQuota Alone Leaves: One Pod Can Eat the Whole Quota

Say the namespace quota is `cpu=2, memory=3G, pods=5, services=3`. A single pod requesting `memory=2G, cpu=1` will **succeed** — it's within the total quota. But now that one pod has consumed the majority of the namespace's entire resource budget, leaving very little for anything else in that namespace. ResourceQuota caps the *total*, but says nothing about how much any *one* pod is allowed to hog.

LimitRange — Bounding What a Single Pod/Container Can Use

LimitRange fixes exactly this gap: it sets **min**, **max**, and **default** CPU/memory *per container*, within a namespace.

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: def-cpu-mem-limit
  namespace: dev
spec:
  limits:
    - default:                  # applied as LIMIT if resources block omits limits
        cpu: 111m
        memory: 99Mi
      defaultRequest:           # applied as REQUEST if resources block omits requests
        cpu: 101m
        memory: 91Mi
      max:                      # hard ceiling
        cpu: 200m
        memory: 100Mi
      min:                      # hard floor
        cpu: 100m
        memory: 90Mi
      type: Container
```

```bash
kubectl describe ns dev
# Resource Limits
# Type        Resource   Min     Max     Default Request   Default Limit   Max Limit/Request
Ratio
# Container   cpu        100m    200m    101m              111m            -
# Container   memory     90Mi    100Mi   91Mi              99Mi            -
```

How the 4 Values Actually Apply

Field	Effect
`min`	Floor — requesting *below* this in `resources.requests` is denied outright
`max`	Ceiling — requesting *above* this in `resources.limits` is denied outright
`default`	Used as the container's **limit** if the Pod YAML's `resources` block omits `limits`
`defaultRequest`	Used as the container's **request** if the Pod YAML's `resources` block omits `requests`

If the Pod YAML has **no `resources:` block at all**, the LimitRange's `default` / `defaultRequest` values get applied automatically (you don't get "unbounded" just because you left it out).

⚠️ **Proof — a request below the minimum gets denied outright**

```bash
kubectl create -f mypod.yaml
# Error from server (Forbidden): error when creating "mypod.yaml": pods "webapp" is
forbidden:
# [minimum memory usage per Container is 90Mi, but request is 80Mi,
#  minimum cpu usage per Container is 100m, but request is 50m]
```

Why This Matters

💡 **The idea behind LimitRange**

No single application should be able to dominate the infrastructure. ResourceQuota protects the namespace's total budget from being exceeded; LimitRange protects that budget from being monopolized by any one pod inside it. They're complementary controls, not alternatives to each other.

✅ **What I Practiced**

- [ ] Created a namespace ResourceQuota (`cpu=2, memory=3G, pods=5, services=3`) and confirmed `kubectl describe ns` shows live `Used` / `Hard` columns.
- [ ] Tried creating a pod with no `resources:` block once a quota existed — confirmed the exact `must specify cpu/memory for` Forbidden error.
- [ ] Deployed 5+ replicas against a `pods=5` quota and read the ReplicaSet's `FailedCreate` events to see quota enforcement happening live.
- [ ] Confirmed a single pod can legally consume most of a namespace's quota on its own, motivating LimitRange.
- [ ] Applied a LimitRange with min/max/default/defaultRequest and confirmed via `kubectl describe ns` that the Resource Limits table matches the YAML.
- [ ] Deliberately requested less than the LimitRange minimum and confirmed the exact denial message naming both the offending resource and the actual vs. minimum values.

Merged into the master reference notes: k8s-master-notes.md §3 (Namespaces & Resource Governance) — "Quota + LimitRange Mechanics, Live" addendum.